UNIVERSITY OF AUCKLAND
Waipapa Taumata Rau
NEW ZEALAND

SCIENCE

COMPSCI 714
Lecture 2: NN and DNN

Neural Networks and Deep Neural Network, what they are and how to train them.

Instructor: Thomas Lacombe

Learning outcomes

- **Explain** the structure and function of artificial neurons, and how they combine to form Neural Networks (NNs) and Deep Neural Networks (DNNs).
- **Describe** how gradient descent and backpropagation enable DNNs to learn from data.

Disclaimer/Acknowledgment:

Some of the following slides are reusing and adapting some of the content created by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley.

All CS188 materials are available at http://ai.berkeley.edu.

Some content is also reused from the CS230 Stanford course slides created by Andrew Ng, available at https://cs230.stanford.edu/syllabus/.

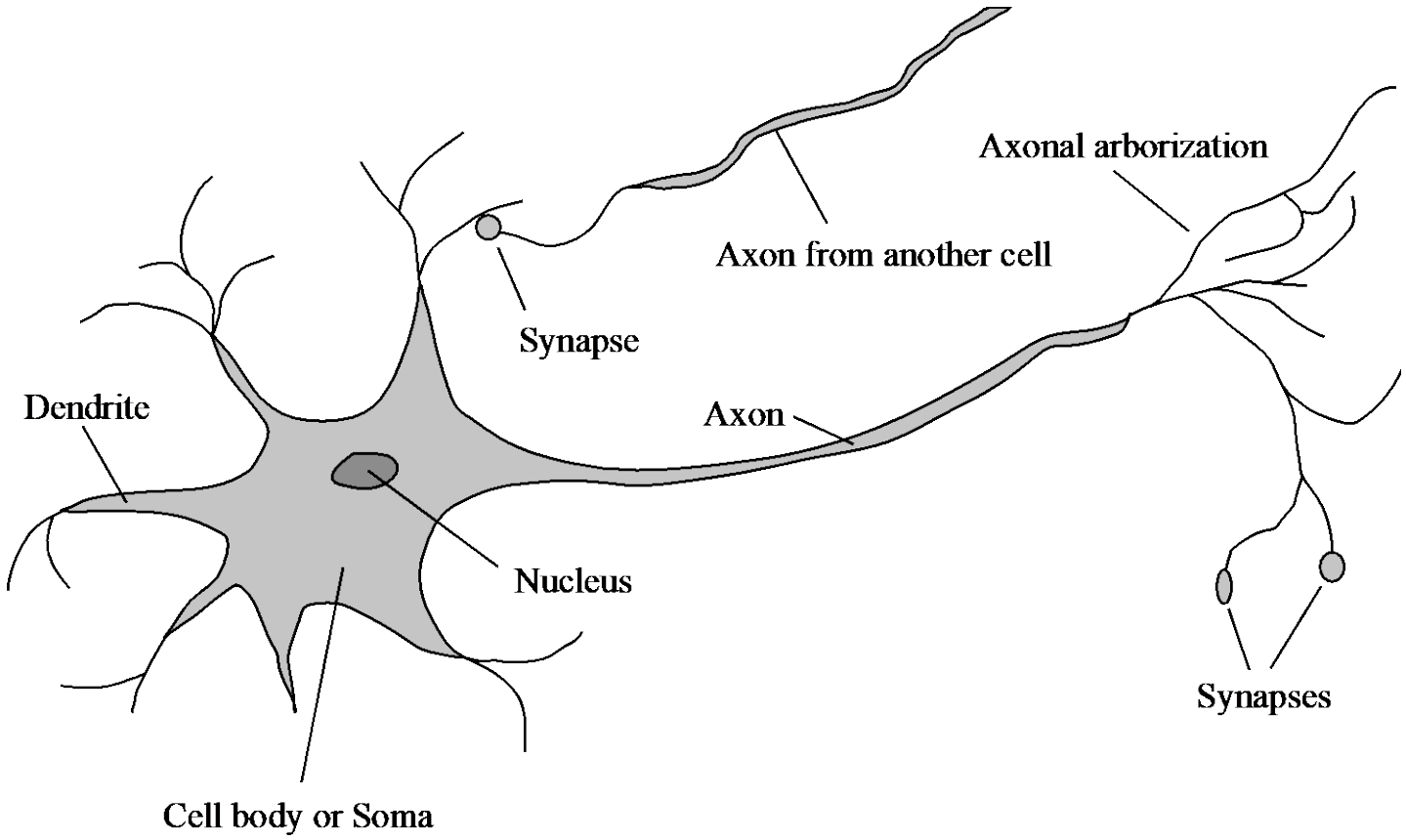
Extra resources

If you want to go further:

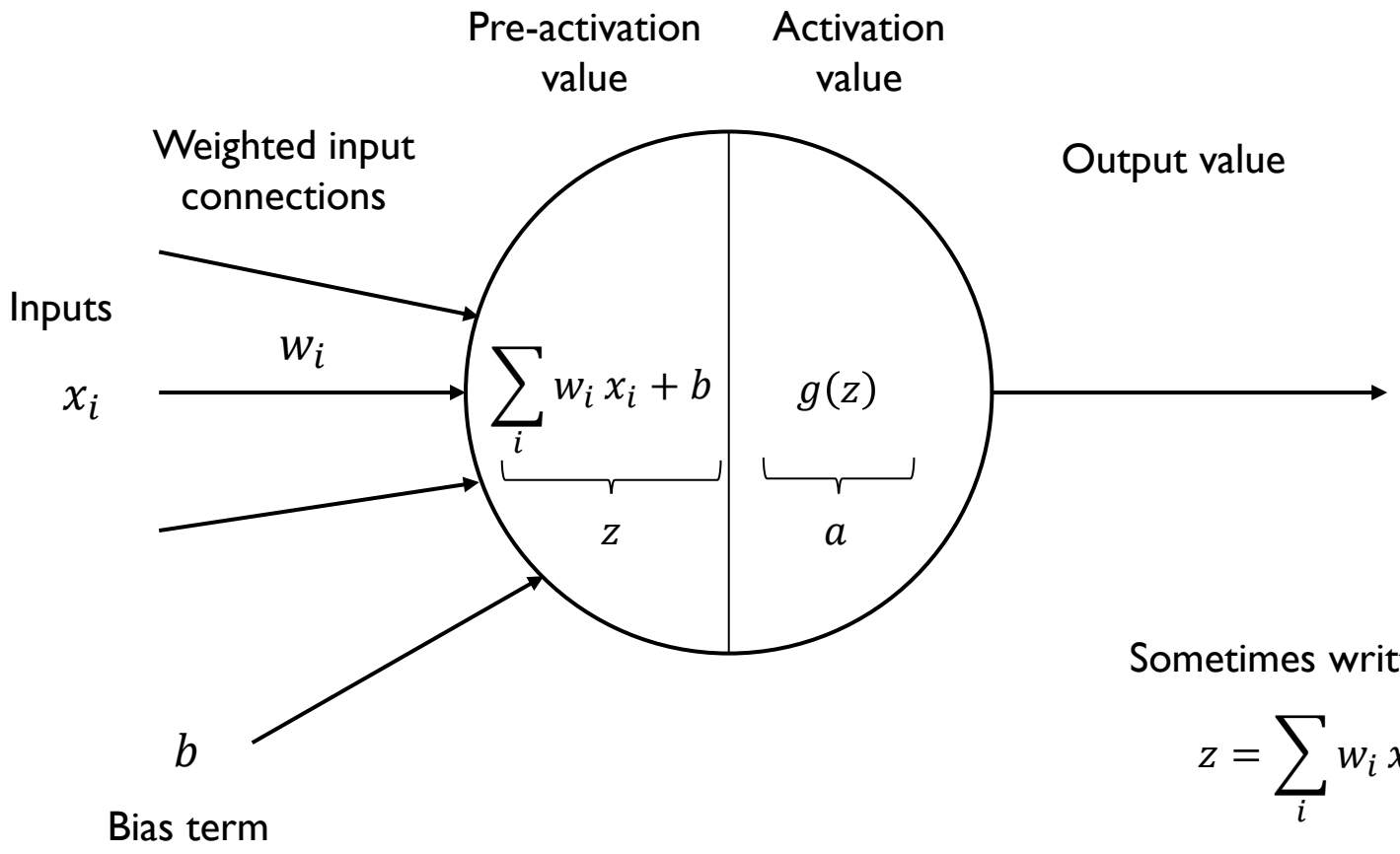
- *Hands-On Machine Learning with Scikit-Learn and PyTorch*, Géron A.
  - Chapters 4, 9, 10 and 11

- *Deep Learning*, Goodfellow I. et al.
  - Part II Deep Networks
  - Part I Applied Math and Machine Learning Basics

Inspiration: Human neurons



Artificial neuron



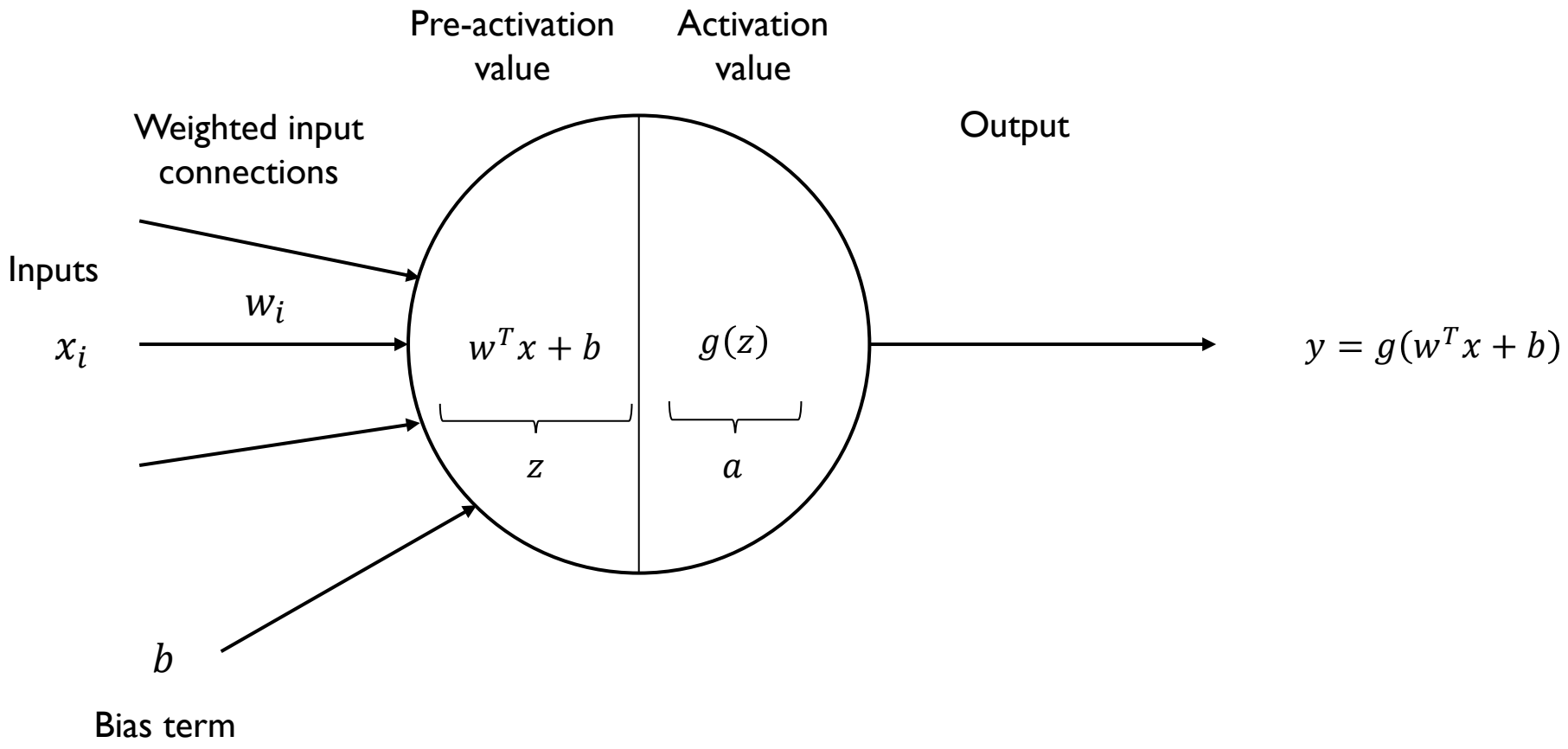
$$y = g\left(\sum_i w_i x_i + b\right)$$

Sometimes written:

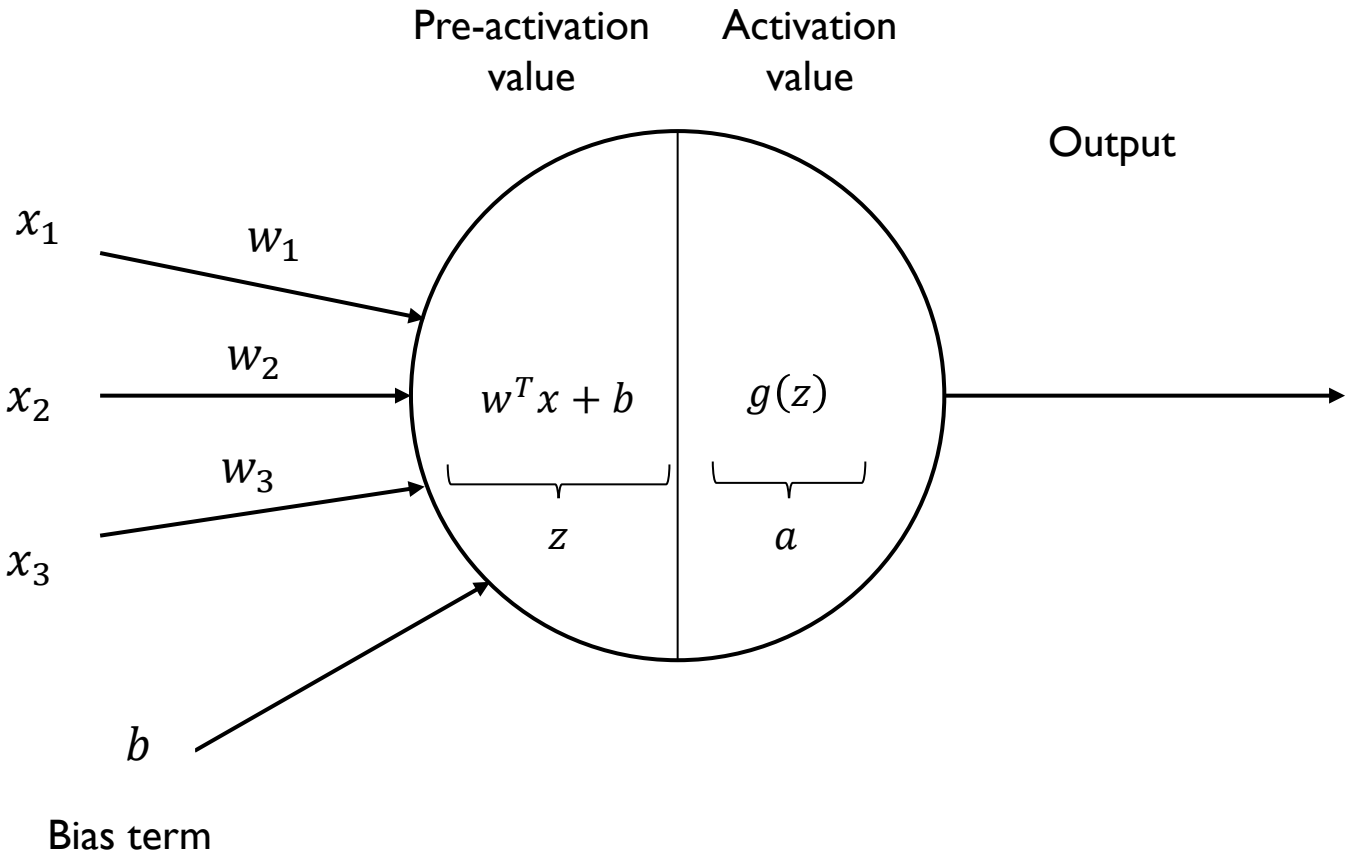
$$z = \sum_i w_i x_i$$

With $x_0 = 1$ with an associated weight w_0 (bias)

Linear algebra representation



Linear algebra representation



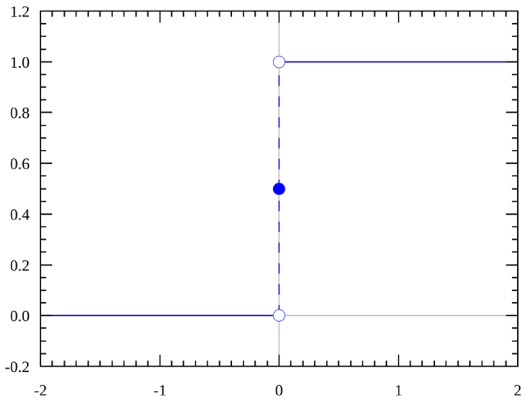
$$y = g\left(\begin{bmatrix} w_1 & w_2 & w_3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + b\right)$$

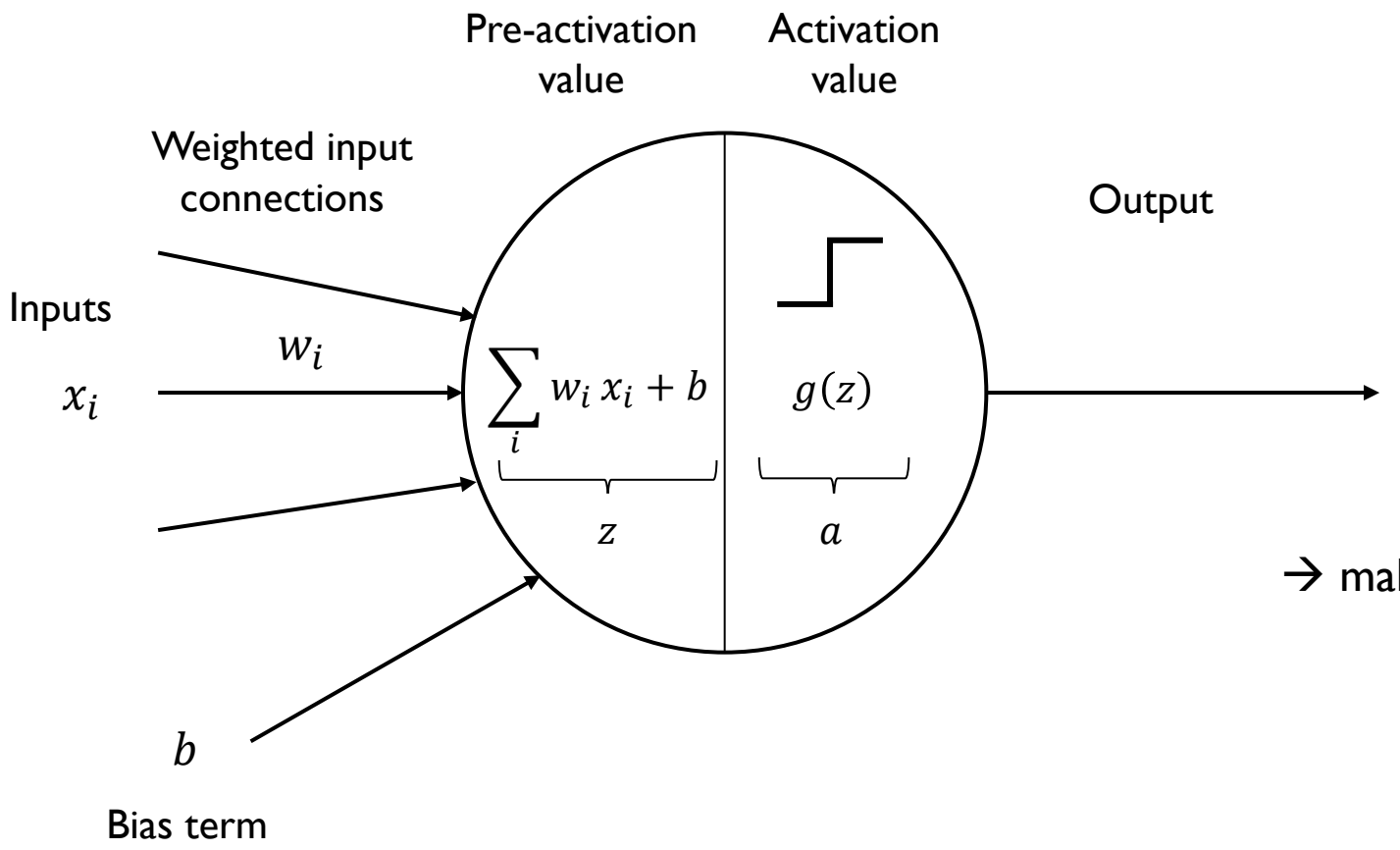
Linear algebra representation

What does the following equation represents?

$$y = g\left(\begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix}\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \end{bmatrix}\right)$$

The Perceptron (Rosenblatt, 1958)





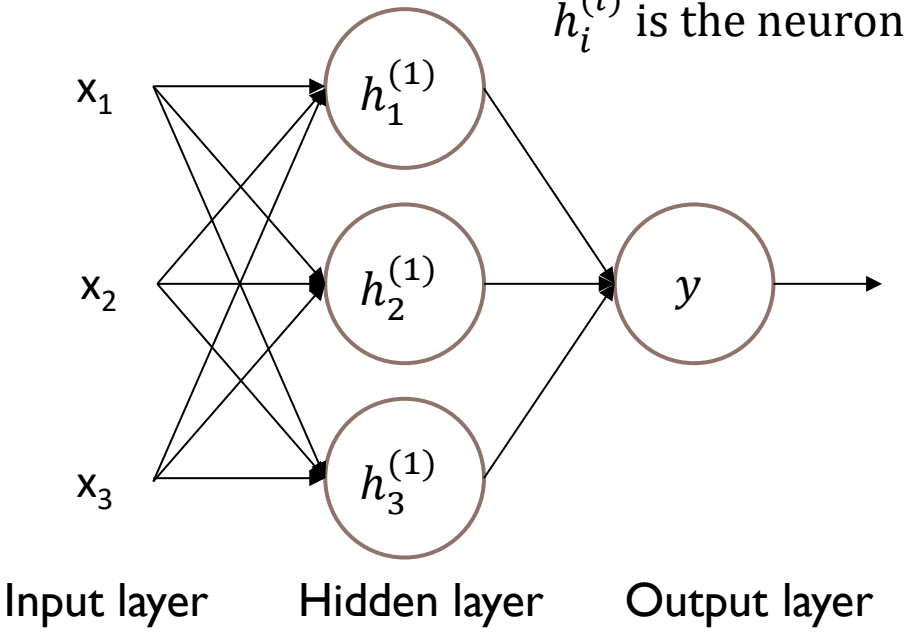
$$y = \begin{cases} 1 & \text{if } \sum_i w_i x_i + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

→ makes a **binary decision** (yes/no, 0/1)

Rosenblatt, F. (1958). *The perceptron: A probabilistic model for information storage and organization in the brain.* **Psychological Review**, 65(6), 386–408.

Multilayer Perceptron (MLP)

$h_i^{(l)}$ is the neuron i in hidden layer l

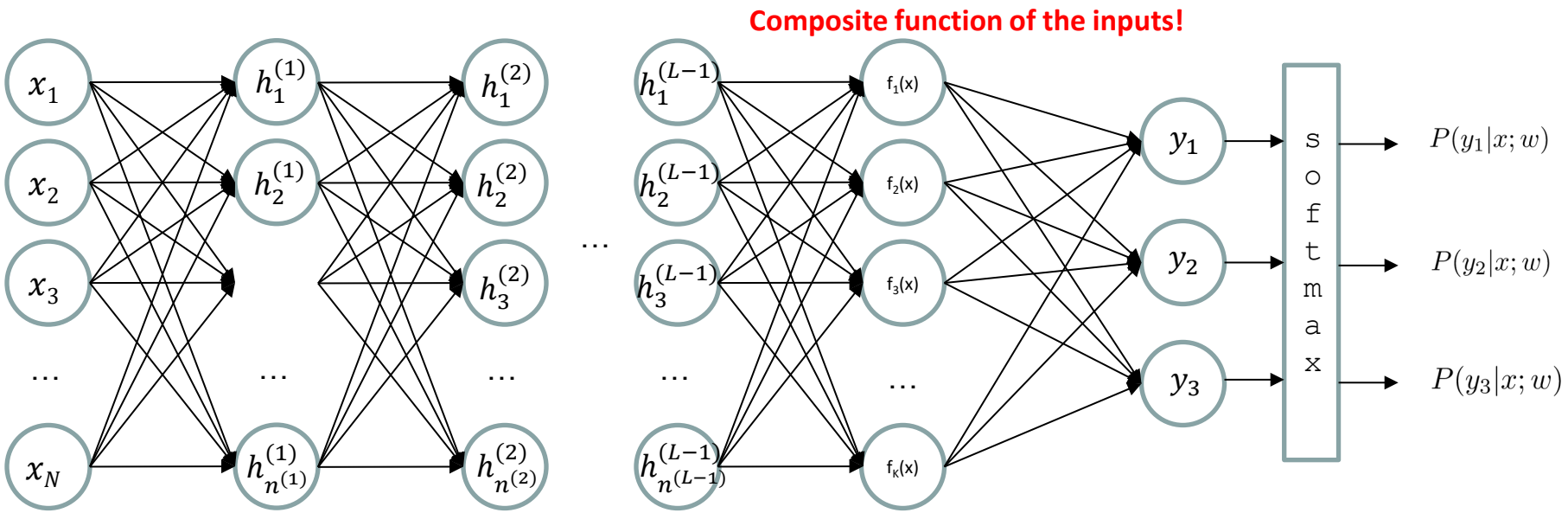


Key properties:

- Fully connected layers
- Feedforward architecture (no loops)
- Uses nonlinear activation functions

Deep Neural Network

- Deep NN = at least 2 hidden layers



Notes

- The NN has $n^{(0)} = N$ inputs and L hidden layers.
- Each hidden layer l has a width of $n^{(l)}$ neurons.
- The output layer has 3 neurons, denoted y_1, y_2 and y_3.
- Softmax is an output activation function applied to output neurons pre-activation values.

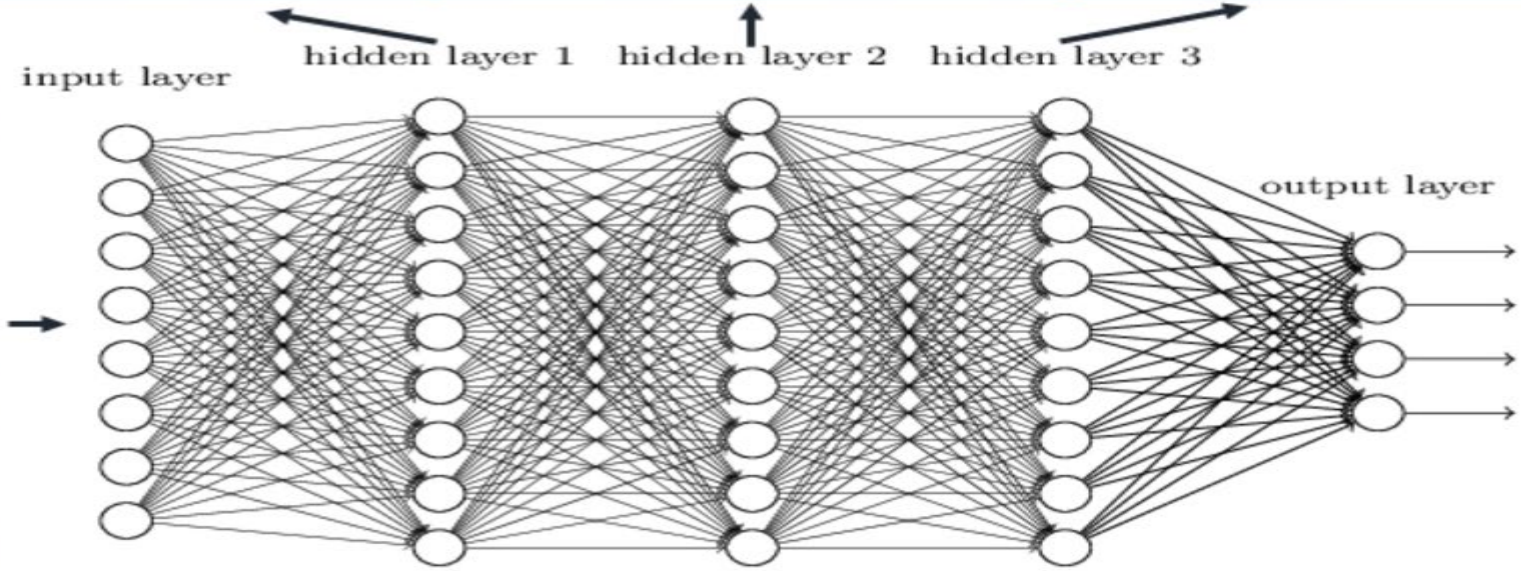
Activation value of a neuron $h_i^{(l)}$:

$$a_i^{(l)} = g\left(z_i^{(l)}\right) = g\left(\sum_j^{n^{(l)}} w_{i,j}^{(l)} a_j^{(l-1)}\right)$$

With $a_j^{(l-1)} = x_j$ when $l = 1$

Intuition about deep features representation

Deep neural networks learn hierarchical feature representations



Inspired from: Le Cun et al. (2015)
Reference: Raphael et al. (2020)

Neural Networks Properties

- **Theorem of the Universal Function Approximators**:

A two-layer neural network (one hidden layer) with a sufficient number of neurons can **approximate** any continuous function to any desired accuracy.

- Practical considerations:
  - Can be seen as learning the features.
  - Large number of neurons:
    - Danger of overfitting

Universal Function Approximation Theorem

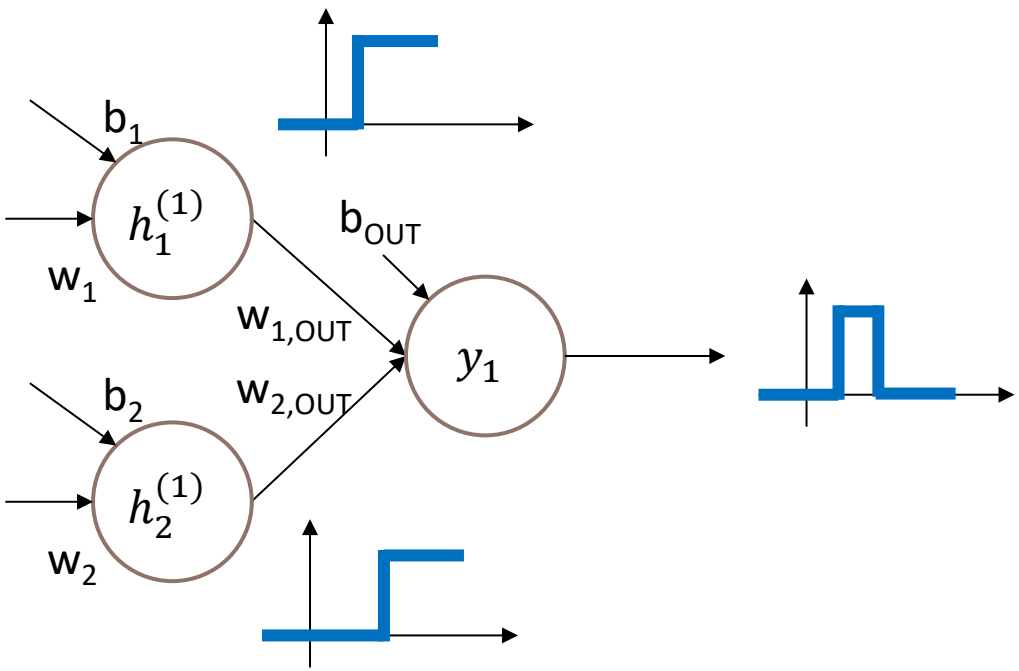
Hornik theorem 1: Whenever the activation function is *bounded and nonconstant*, then, for any finite measure μ, standard multilayer feedforward networks can approximate any function in $L^p(\mu)$ (the space of all functions on R^k such that $\int_{R^k} |f(x)|^p d\mu(x) < \infty$) arbitrarily well, provided that sufficiently many hidden units are available.

Hornik theorem 2: Whenever the activation function is *continuous, bounded and nonconstant*, then, for arbitrary compact subsets $X \subseteq R^k$, standard multilayer feedforward networks can approximate any continuous function on X arbitrarily well with respect to uniform distance, provided that sufficiently many hidden units are available.

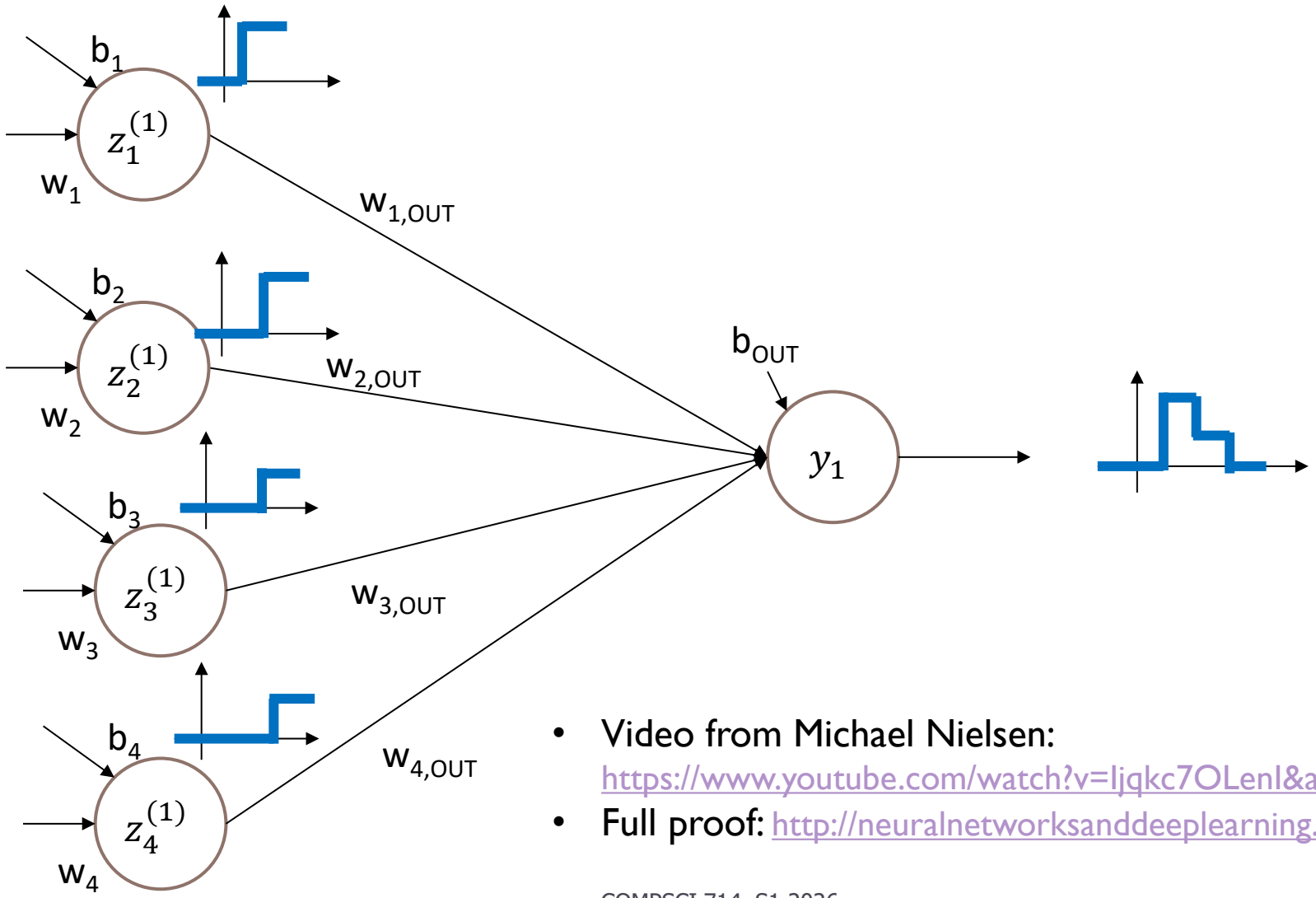
- In simple words: Given any continuous function f(x), if a 2-layer neural network has enough hidden units, then there is a choice of weights that allow it to closely approximate f(x).

Cybenko (1989) “Approximations by superpositions of sigmoidal functions”
Hornik (1991) “Approximation Capabilities of Multilayer Feedforward Networks”
Leshno and Schocken (1991) ”Multilayer Feedforward Networks with Non-Polynomial Activation Functions Can Approximate Any Function”

Intuition for the theorem



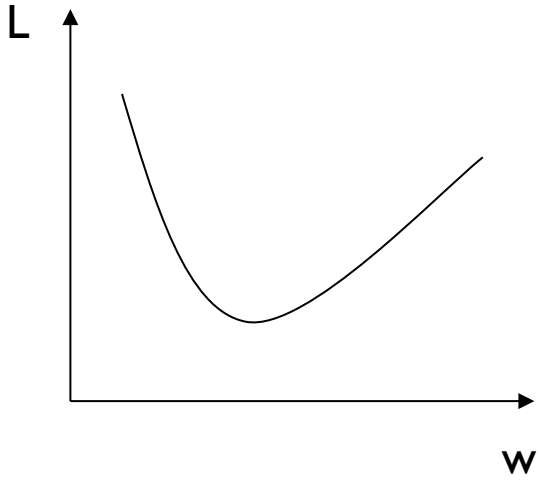
Intuition for the theorem



- Video from Michael Nielsen: https://www.youtube.com/watch?v=Ijqkc7OLenI&ab_channel=MichaelNielsen
- Full proof: http://neuralnetworksanddeeplearning.com/chap4.html

Learning a function is an optimisation problem

- Optimisation of the parameters (weights) to minimise a cost/loss/error function



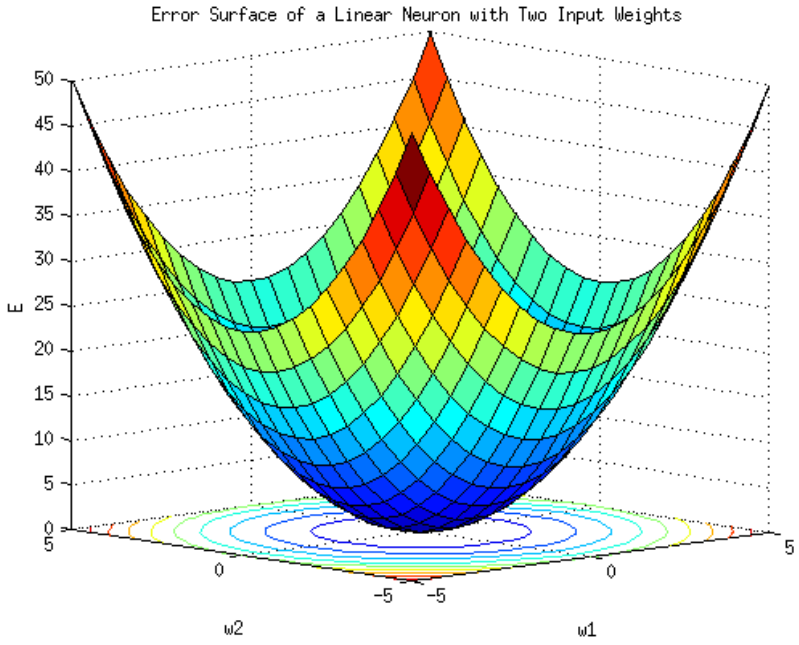


Figure source: https://srinivas-yeeda.medium.com/loss-functions-in-deep-learning-models-129866be93e

Gradient Descent

- Perform update in downhill direction for each coordinate.
- The steeper the slope (i.e. the higher the derivative) the bigger the step for that coordinate.
- E.g., consider: $g(w_1, w_2)$
  - Updates:

$$w_1 \leftarrow w_1 - \alpha * \frac{\partial g}{\partial w_1}(w_1, w_2)$$

$$w_2 \leftarrow w_2 - \alpha * \frac{\partial g}{\partial w_2}(w_1, w_2)$$

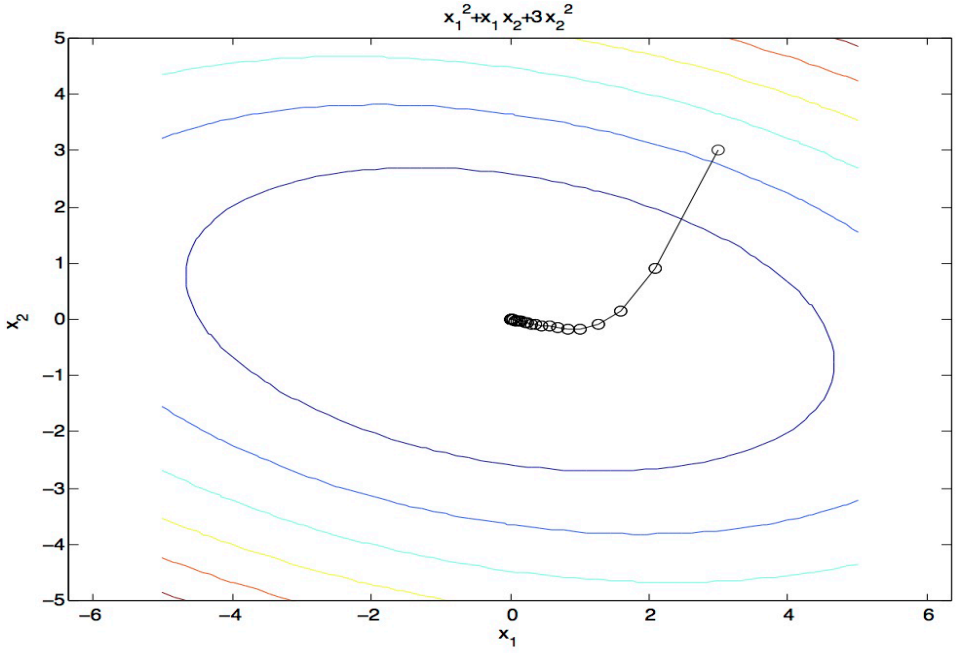
- Updates in vector notation:

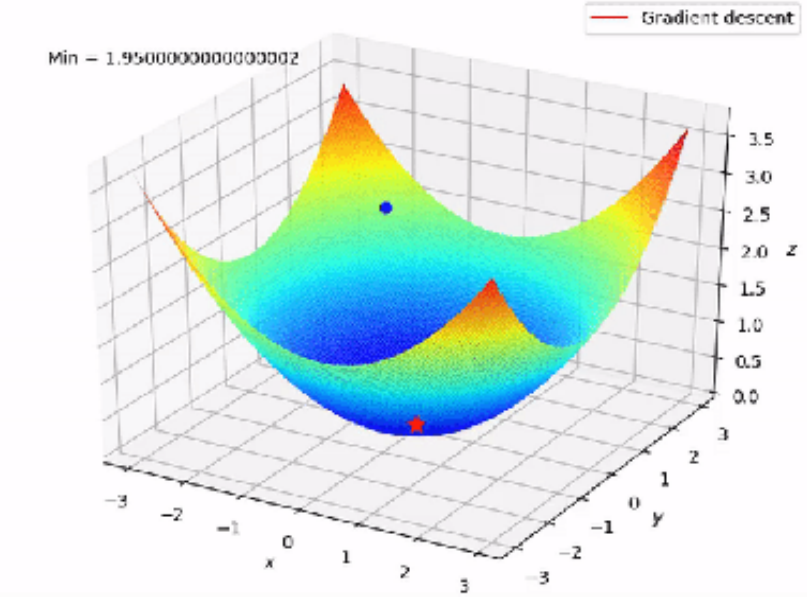
$$w \leftarrow w - \alpha * \nabla_w g(w)$$

with: $\nabla_w g(w) = \begin{bmatrix} \frac{\partial g}{\partial w_1}(w) \\ \frac{\partial g}{\partial w_2}(w) \end{bmatrix}$ **= gradient**

Steepest Descent

- Idea:
  - Start somewhere
  - Repeat: Take a step in the steepest descent direction





Figures source: Mathworks and https://suniljangirblog.wordpress.com/2018/12/03/the-outline-of-gradient-descent/

Steepest Direction

- Steepest Direction = direction of the gradient

$$\nabla g = \begin{bmatrix} \frac{\partial g}{\partial w_1} \\ \frac{\partial g}{\partial w_2} \\ \cdots \\ \frac{\partial g}{\partial w_n} \end{bmatrix}$$

Optimisation Procedure: Gradient Descent

- `init` w
- `for iter = 1, 2, …`

$$w \leftarrow w - \alpha * \nabla g(w)$$

- α : learning rate --- hyperparameter that needs to be chosen carefully

- If too high → chance to miss the optimum
- If too low → very long time

Gradient Descent / Stochastic GD / Mini-batch GD

- Gradient Descent (GD) / Batch GD: updates weights based on loss over the full training data (batch size = dataset size)
  - → Smooth descent as using average gradient over training set (true gradient)
  - → Can be used for small datasets (e.g., max. 2000 instances)

- Stochastic GD (SGD): updates weights based on loss over a randomly chosen instance of data (1 datapoint / iteration)
  - → More efficient than GD but fluctuates a lot

- Mini-batch GD: updates weights based on loss over a randomly chosen subset of data (>1 datapoint / iteration)
  - → More efficient than GD and less fluctuations than SGD
  - → Typical batch sizes: 64, 128, 256, 512, … (fit CPU/GPU memory)

Gradient Descent / Stochastic GD, Mini-batch GD

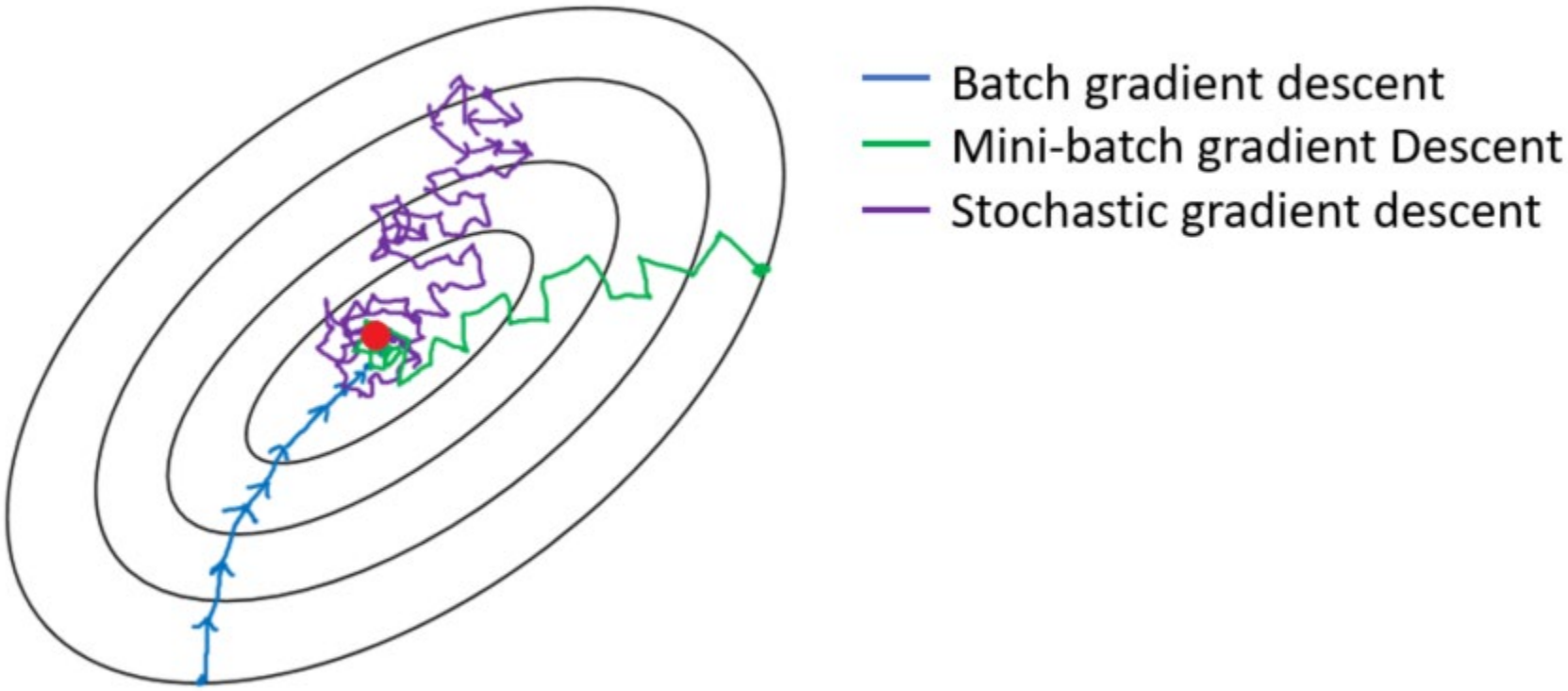


Figure source: https://medium.com/analytics-vidhya/gradient-descent-vs-stochastic-gd-vs-mini-batch-sgd-fbd3a2cb4ba4

Training a full network

- Gradient descent = how parameters are updated
- How to calculate the updates, i.e., the gradients, in MLPs?

- This was solved in the 1980's by applying the **backpropagation method** (1960's) to training MLPs.

$$w \leftarrow w - \alpha * \nabla_w g(w)$$

with: $\nabla_w g(w) = \begin{bmatrix} \frac{\partial g}{\partial w_1}(w) \\ \frac{\partial g}{\partial w_2}(w) \end{bmatrix}$ **= gradient**

How about computing all the derivatives?

- Derivatives tables:

$$\frac{d}{dx}(a) = 0$$

$$\frac{d}{dx}(x) = 1$$

$$\frac{d}{dx}(au) = a\frac{du}{dx}$$

$$\frac{d}{dx}(u+v-w) = \frac{du}{dx} + \frac{dv}{dx} - \frac{dw}{dx}$$

$$\frac{d}{dx}(uv) = u\frac{dv}{dx} + v\frac{du}{dx}$$

$$\frac{d}{dx}\left(\frac{u}{v}\right) = \frac{1}{v}\frac{du}{dx} - \frac{u}{v^2}\frac{dv}{dx}$$

$$\frac{d}{dx}(u^n) = nu^{n-1}\frac{du}{dx}$$

$$\frac{d}{dx}(\sqrt{u}) = \frac{1}{2\sqrt{u}}\frac{du}{dx}$$

$$\frac{d}{dx}\left(\frac{1}{u}\right) = -\frac{1}{u^2}\frac{du}{dx}$$

$$\frac{d}{dx}\left(\frac{1}{u^n}\right) = -\frac{n}{u^{n+1}}\frac{du}{dx}$$

$$\frac{d}{dx}[f(u)] = \frac{d}{du}[f(u)]\frac{du}{dx}$$

$$\frac{d}{dx}[\ln u] = \frac{d}{dx}[\log_e u] = \frac{1}{u}\frac{du}{dx}$$

$$\frac{d}{dx}[\log_a u] = \log_a e\frac{1}{u}\frac{du}{dx}$$

$$\frac{d}{dx}e^u = e^u\frac{du}{dx}$$

$$\frac{d}{dx}a^u = a^u \ln a\frac{du}{dx}$$

$$\frac{d}{dx}(u^v) = vu^{v-1}\frac{du}{dx} + \ln u \;\; u^v\frac{dv}{dx}$$

$$\frac{d}{dx}\sin u = \cos u\frac{du}{dx}$$

$$\frac{d}{dx}\cos u = -\sin u\frac{du}{dx}$$

$$\frac{d}{dx}\tan u = \sec^2 u\frac{du}{dx}$$

$$\frac{d}{dx}\cot u = -\csc^2 u\frac{du}{dx}$$

$$\frac{d}{dx}\sec u = \sec u\tan u\frac{du}{dx}$$

$$\frac{d}{dx}\csc u = -\csc u\cot u\frac{du}{dx}$$

Image source: http://hyperphysics.phy-astr.gsu.edu/hbase/Math/derfunc.html

How about computing all the derivatives?

- But what if neural net function is not in the table?
  - No problem: CHAIN RULE: $$a_i^{(l)} = \mathrm{g}\left(\sum_j^{n^{(l)}} w_{i,j}^{(l)} a_j^{(l-1)}\right)$$
    - If $$f(x) = g(h(x))$$
    - Then $$f'(x) = g'(h(x))h'(x)$$
- **Naively computing all the gradients have an exponential computational cost** for deep networks!

Backpropagation

- **Backpropagation key idea:** reuse intermediate results and share partial derivatives efficiently to keep the computational cost close to a forward pass.

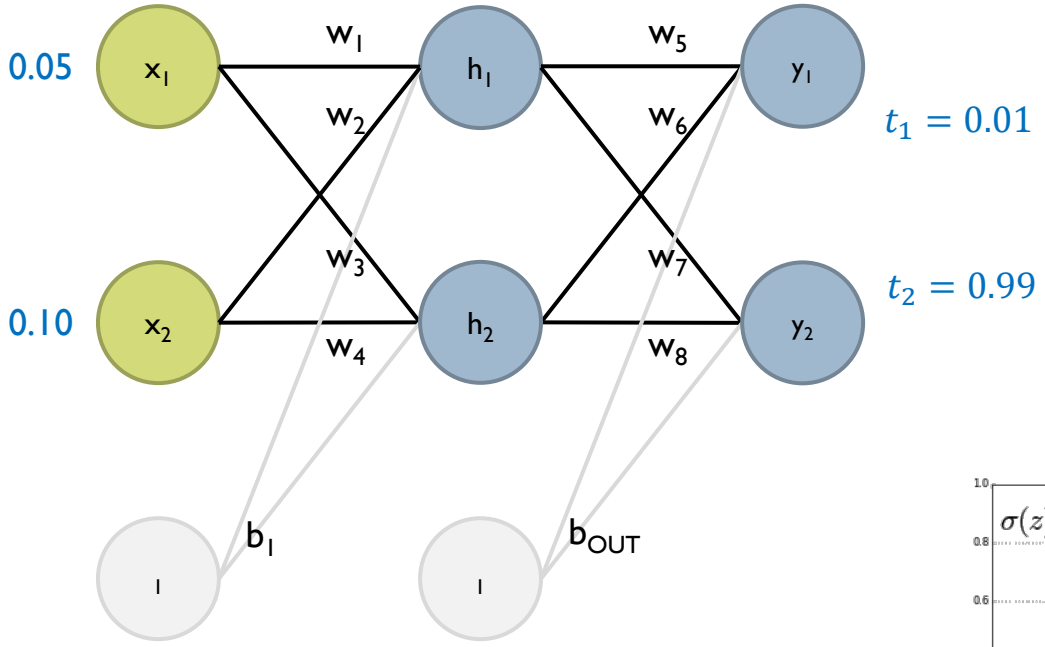
- **Backpropagation key steps:**
  1) Apply the chain rule of calculus
  2) Propagate error signal backward from output layer to early layers
  3) Efficiently computes gradient of loss w.r.t. every weight and bias in the network

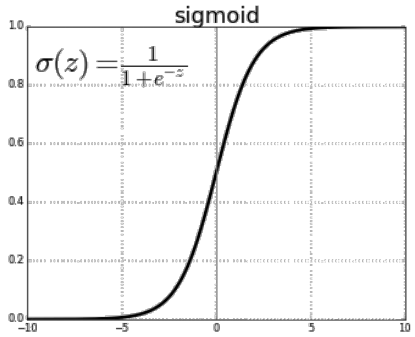
Automatic Differentiation

- Automatic differentiation software/tools
  - e.g.: Theano, TensorFlow, PyTorch, Chainer.
  - Only need to program the function g(x,y,w).
  - Can automatically compute all derivatives w.r.t. all entries in w.
  - This is typically done by caching info during forward computation pass and then doing a backward pass = “backpropagation”.
  - Autodiff / Backpropagation can often be done at computational cost comparable to the forward pass.

- If you want more details, you can read Appendix A of the “Hands on ML” book.

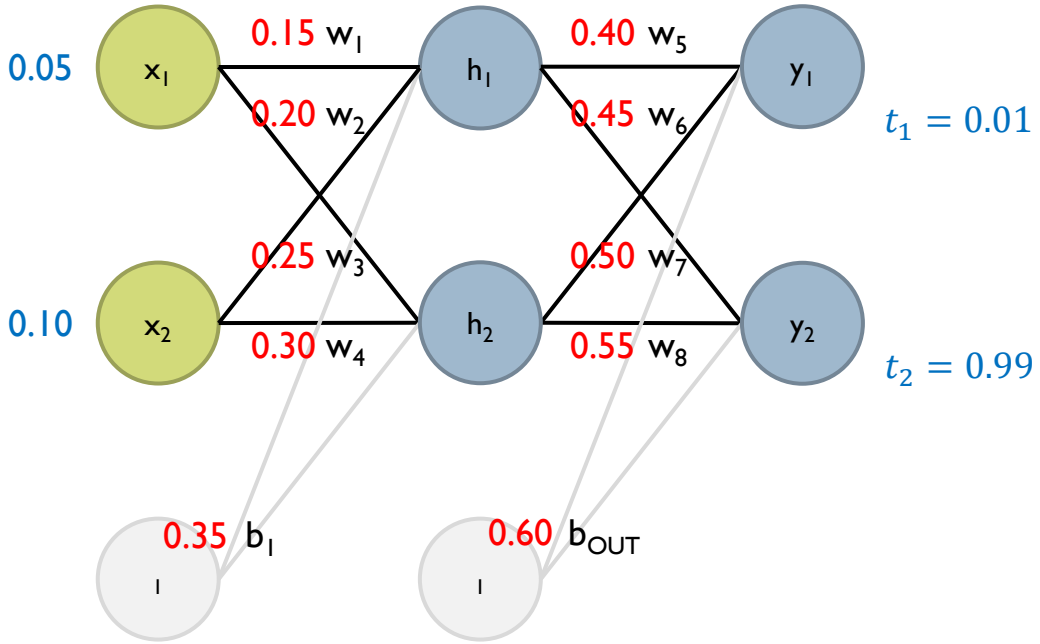
Backpropagation in a NN





Backpropagation in a NN

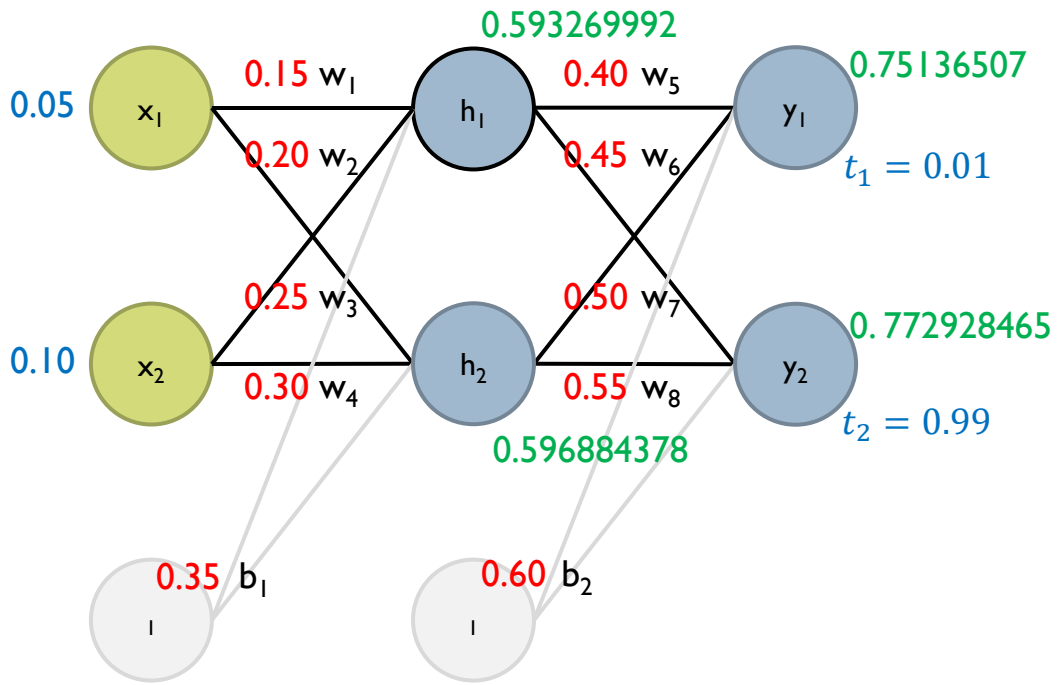
- Initialisation of the weights



Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

- Forward pass



$$h_1 = S(\overbrace{w_1 * x_1 + w_2 * x_2 + b_1 * 1}^{z_{h_1}})$$
$$= S(0.15 * 0.05 + 0.20 * 0.10 + 0.35 * 1)$$
$$= S(0.3775)$$
$$= 0.593269992$$

$$h_2 = S(\overbrace{w_3 * x_1 + w_4 * x_2 + b_1 * 1}^{z_{h_2}})$$
$$= S(0.25 * 0.05 + 0.30 * 0.10 + 0.35 * 1)$$
$$= 0.596884378$$

$$y_1 = S(\overbrace{w_5 * h_1 + w_6 * h_2 + b_2 * 1}^{z_{y_1}})$$
$$= 0.75136507$$

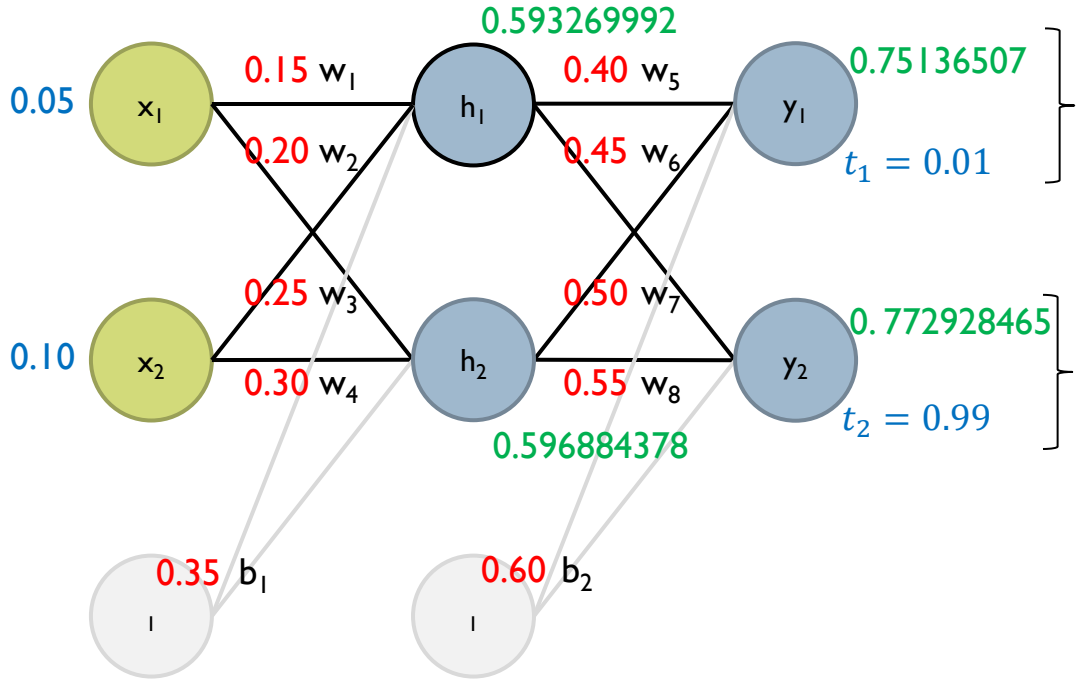
$$y_2 = S(\overbrace{w_7 * h_1 + w_8 * h_2 + b_2 * 1}^{z_{y_2}})$$
$$= 0.772928465$$

Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

- Calculating the total error

$$E(y, y') = \frac{1}{2}(y - \hat{y})^2$$



$$E_{y_1} = \frac{1}{2}(0.01 - 0.75136507)^2$$
$$= 0.274811083$$

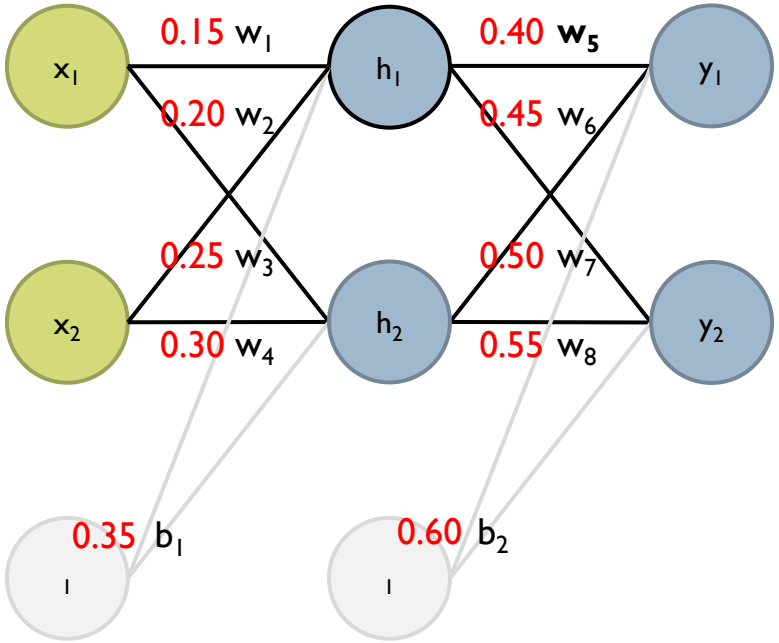
$$E_{y_2} = \frac{1}{2}(0.99 - 0.772928465)^2$$
$$= 0.023560026$$

$$E_{total} = E_{y_1} + E_{y_2}$$
$$= 0.298371109$$

Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

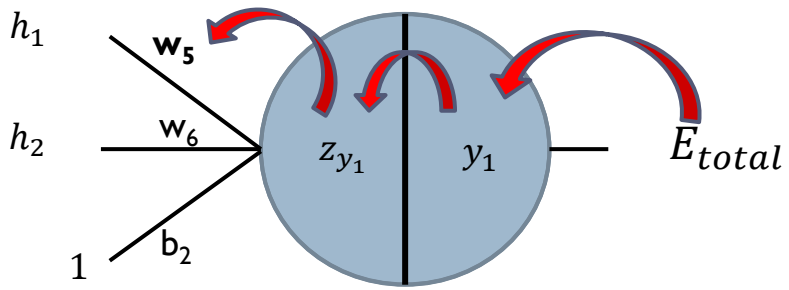
- Back propagation (output layer)



$\mathbf{w_5}$ influence on E_{total}? $\frac{\partial E_{total}}{\partial \mathbf{w}_5}$

Chain rule:

$$\frac{\partial E_{total}}{\partial \mathbf{w}_5} = \frac{\partial E_{total}}{\partial y_1} * \frac{\partial y_1}{\partial z_{y_1}} * \frac{\partial z_{y_1}}{\partial \mathbf{w}_5}$$



Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

- Back propagation (output layer)

$$\frac{\partial E_{total}}{\partial \mathbf{w}_5} = \frac{\partial E_{total}}{\partial y_1} * \frac{\partial y_1}{\partial z_{y_1}} * \frac{\partial z_{y_1}}{\partial \mathbf{w}_5}$$

$$\begin{aligned} E_{total} &= E_{y_1} + E_{y_2} \\ &= \frac{1}{2}(t_1 - y_1)^2 + \frac{1}{2}(t_2 - y_2)^2 \end{aligned}$$

$$\begin{aligned} \frac{\partial E_{total}}{\partial y_1} &= 2 * \frac{1}{2}(t_1 - y_1)^{2-1} * -1 + 0 \\ &= y_1 - t_1 \\ &= 0.75136507 - 0.01 \\ &= 0.74136507 \end{aligned}$$

Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

- Back propagation (output layer)

$$\frac{\partial E_{total}}{\partial \mathrm{w}_5} = \frac{\partial E_{total}}{\partial y_1} * \frac{\partial y_1}{\partial z_{y_1}} * \frac{\partial z_{y_1}}{\partial \mathrm{w}_5}$$

$$y_1 = \frac{1}{1+ e^{-z_1{}^{(2)}}}$$ (activation = sigmoid function)

Sigmoid → g'(x) = g(x)(1-g(x))

$$\frac{\partial y_1}{\partial z_1{}^{(2)}} = y_1 * (1 - y_1)$$
$$= 0.75136507 * (1 - 0.75136507)$$
$$= 0.186815602$$

Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

- Back propagation (output layer)

$$\frac{\partial E_{total}}{\partial \mathrm{w}_5} = \frac{\partial E_{total}}{\partial y_1} * \frac{\partial y_1}{\partial z_{y_1}} * \frac{\partial z_{y_1}}{\partial \mathrm{w}_5}$$

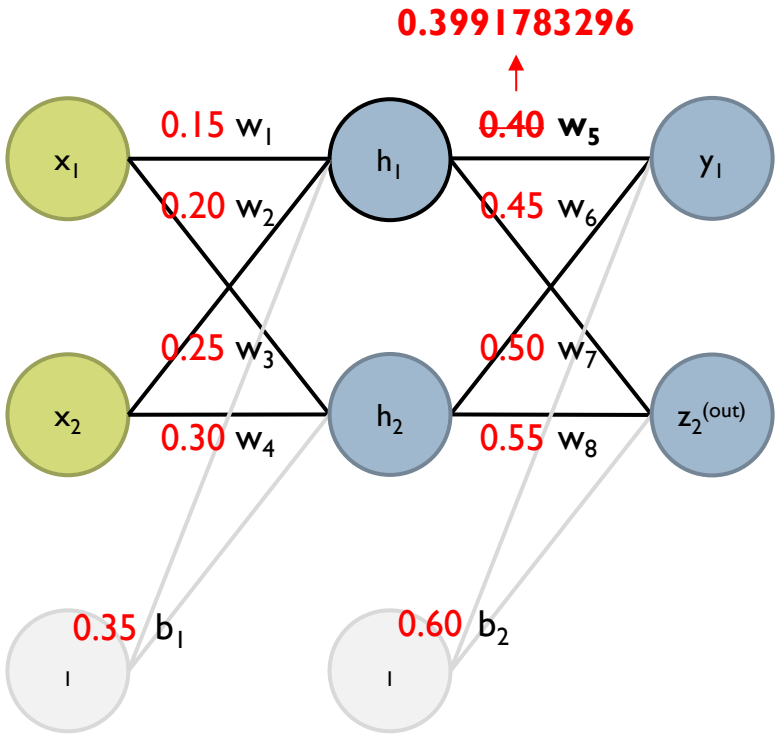
$$z_{y_1} = \mathrm{w}_5 * h_1 + \mathrm{w}_6 * h_2 + \mathrm{b}_2 * 1$$

$$\begin{aligned}\frac{\partial z_{y_1}}{\partial \mathrm{w}_5} &= h_1 + 0 + 0 \\ &= h_1 \\ &= 0.593269992\end{aligned}$$

Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

- Back propagation (output layer)



$$\frac{\partial E_{total}}{\partial \mathbf{w_5}} = \frac{\partial E_{total}}{\partial y_1} * \frac{\partial \boldsymbol{y_1}}{\partial \boldsymbol{z_1}^{(2)}} * \frac{\partial z_1{}^{(2)}}{\partial \mathbf{w_5}}$$

$$= 0.74136507 * 0.186815602 * 0.593269992$$
$$= 0.082167041$$

Gradient descent to decrease the error
(with learning rate $\alpha = 0.01$) :

$$\mathrm{w}_5 \leftarrow \mathrm{w}_5 - \alpha * \frac{\partial E_{total}}{\partial \mathbf{w_5}}$$

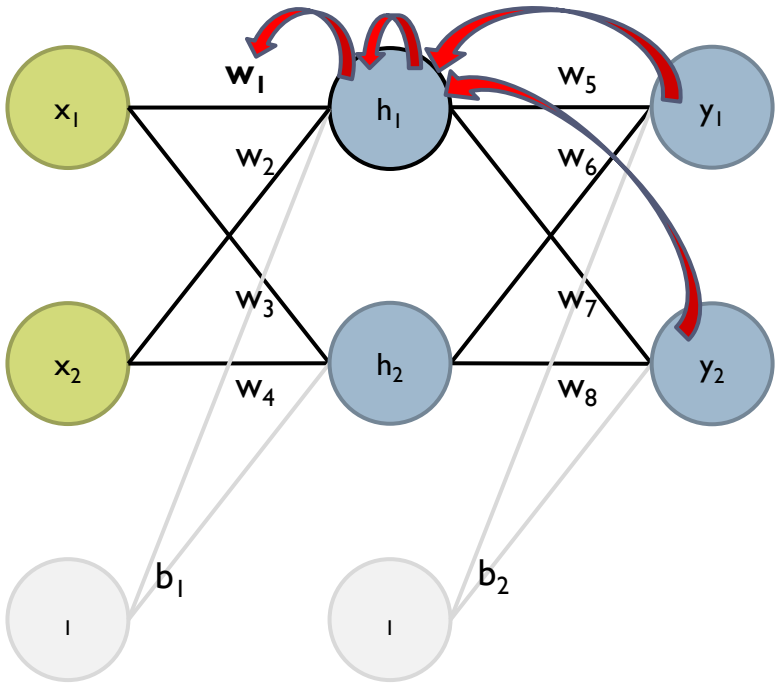
$$\mathrm{w}_5 \leftarrow 0.40 - 0.01 * 0.082167041$$

$$\mathrm{w}_5 \leftarrow 0.3991783296$$

Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

- Back propagation



$$\frac{\partial E_{total}}{\partial w_1} = \underbrace{\frac{\partial E_{total}}{\partial h_1}} * \frac{\partial h_1}{\partial z_{h_1}} * \frac{\partial z_{h_1}}{\partial w_1}$$

$$\frac{\partial E_{total}}{\partial y_1} = \frac{\partial E_{y_1}}{\partial y_1} + \frac{\partial E_{y_2}}{\partial y_2}$$

Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Conclusion

IT'S FRIDAY NIGHT, ALL
MY FRIENDS ARE AT THE PARTY
BUT, I'M HERE TRYING TO
FIGURE OUT BACKPROPAGATION
imgflip.com